

Variational Quantum Eigensolver (VQE) — Detailed Notes (Session 9)

Course: CS490/5590 — Quantum Computing Applications in Data Science, AI, & Deep Learning

Instructor: Luke Miller

0) Introduction

Goal. Given a **Hamiltonian** H (a Hermitian linear operator), estimate its **ground-state energy** E_0 (the smallest eigenvalue, a real number) by preparing a parameterised quantum state $|\psi(\theta)\rangle$ (a complex column vector; θ is a real parameter vector) and minimising the **energy** $E(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle$ (a real-valued function).

Why this matters. Many problems in **chemistry** and **optimisation** reduce to “find the lowest energy” of some model. Quantum devices are natural samplers of expectation values $\langle P \rangle = \langle \psi | P | \psi \rangle$ for **Pauli strings** P (tensor products of I, X, Y, Z), and classical computers are excellent at noisy, real-valued optimisation. VQE combines both.

1) VQE in brief: what it does and the steps

What VQE is. A hybrid algorithm that alternates between preparing $|\psi(\theta)\rangle$ on qubits and classically updating θ to reduce $E(\theta)$.

Steps.

1. **Model to qubits.** Express the problem as a qubit-space Hamiltonian $H = \sum_i h_i P_i$ where h_i are real coefficients and P_i are Pauli strings.
2. **Pick an ansatz.** Choose a circuit family $U(\theta)$ with parameters $\theta \in \mathbb{R}^d$ (a real vector of length d); prepare $|\psi(\theta)\rangle = U(\theta)|0 \cdots 0\rangle$.
3. **Measure terms.** Estimate $\langle P_i \rangle = \langle \psi(\theta) | P_i | \psi(\theta) \rangle$ and compute $E(\theta) = \sum_i h_i \langle P_i \rangle$.
4. **Update parameters.** Use a classical optimiser to map $\theta \mapsto \theta'$ that lowers the real number $E(\theta)$.
5. **Repeat** until a stopping rule holds.

Why these steps.

- Step 1 makes the target objective compatible with qubits.
- Step 2 restricts search to states a device can prepare at low depth.
- Step 3 gathers the information that defines the cost.
- Step 4 performs progress using mature optimisation methods.

2) The VQE process, step by step

2.1 What an Ising model is, in practical terms, and why we use it

Spin view. An Ising model matches problems with binary choices. Each choice is a spin $s_i \in \{-1, +1\}$ for index i (an integer). A configuration $s = (s_1, \dots, s_n)$ for n spins (a positive integer) has energy

$$E_{\text{Ising}}(s) = \sum_i h_i s_i + \sum_{i < j} J_{ij} s_i s_j,$$

where h_i are real "biases" for individual spins and J_{ij} are real "couplings" that reward or penalise pairs being aligned ($s_i s_j = +1$) or anti-aligned ($s_i s_j = -1$).

Qubit view. Replace s_i by the eigenvalues of **Pauli- Z** on qubit i . The Ising Hamiltonian becomes

$$H_{\text{Ising}} = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j + c,$$

where Z_i is the Pauli- Z matrix on qubit i (the 2×2 matrix $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ applied to qubit i), $Z_i Z_j$ is their tensor product, and c is a real constant shift.

Why Ising.

- It **encodes discrete objectives** (feature selection, max-cut, scheduling) using only Z and ZZ terms, which are easy to measure.
- It **bridges** to chemistry and other models because any qubit Hamiltonian can be written as a **Pauli sum**; Ising is the simplest nontrivial case and is directly usable by VQE.
- It **separates modelling from solving**: once you have H_{Ising} , VQE can target $\min_{\theta} \langle \psi(\theta) | H_{\text{Ising}} | \psi(\theta) \rangle$.

2.2 Building Hamiltonians: chemistry and mapping to Pauli sums

Second-quantised chemistry.

$$H = \sum_{p,q} h_{pq} a_p^\dagger a_q + \frac{1}{2} \sum_{p,q,r,s} h_{pqrs} a_p^\dagger a_q^\dagger a_r a_s.$$

Here $a_p^\dagger, a_p$ are fermionic creation/annihilation operators; indices p, q, r, s label spin-orbitals (integers); h_{pq}, h_{pqrs} are real coefficients from integrals.

Mapping to qubits.

$$H = \sum_i h_i P_i, \quad P_i \in \{I, X, Y, Z\}^{\otimes n}.$$

Each P_i is a tensor product of I, X, Y, Z acting on n qubits; h_i are real coefficients.

Practical choices: Jordan–Wigner, Parity, Bravyi–Kitaev.

Why Pauli sums.

- They are the **native language** of qubit devices: we can both **prepare** states using exponentials $e^{-i(\theta/2)P}$ and **measure** $\langle P \rangle$ by basis changes and counts.
- They make $E(\theta) = \sum_i h_i \langle P_i \rangle$ a **linear** combination of observed averages, which keeps variance accounting simple.

Example (H_2 , STO-3G).

$$H = (-1.05) Z_0 + 0.39 Z_1 + 0.18 Z_0 Z_1 + 0.55 X_0 X_1 + 0.55 Y_0 Y_1.$$

Here Z_0 means Z on qubit 0, X_1 means X on qubit 1, etc.; coefficients are real numbers.

2.3 How Pauli operators encode circuits and why we permute them

Single-qubit Pauli matrices.

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix},$$

with $i^2 = -1$ for the imaginary unit i .

From a string to a gate. A Pauli string P (for example $Z_i Z_j$ on qubits i and j) generates a unitary rotation

$$U_P(\theta) = \exp\left(-i \frac{\theta}{2} P\right),$$

where θ is a real rotation angle. This describes both **ansatz layers** and **time-evolution-style gates**.

Compilation pattern. To implement $U_P(\theta)$ on hardware:

1. **Basis change** so that the active Pauli on each affected qubit becomes Z . For example, to treat an X as a Z , use the Hadamard H because $H^\dagger Z H = X$; to treat a Y as a Z , use $S^\dagger H$ because $(S^\dagger H)^\dagger Z (S^\dagger H) = Y$.
2. CNOT chain to collect parity onto one qubit for multi-qubit strings.
3. Single $R_Z(\theta)$ rotation on that qubit, where $R_Z(\theta) = e^{-i(\theta/2)Z}$.
4. Uncompute the CNOT chain and reverse the basis change.

Why we permute Pauli strings (and why basis changes).

- **Measurement efficiency.** If two strings commute, they can be measured in the same basis. By permuting/conjugating strings into shared Z -aligned forms, we reduce the number of distinct circuits.
- **Hardware alignment.** Some devices prefer Z -basis rotations. Conjugating X or Y into Z aligns with native gates, reduces depth, and improves fidelity.
- **Noise control.** Fewer basis settings and shorter CNOT chains mean less accumulated error for the same $\langle P \rangle$ precision.

Measuring P as Z . To estimate $\langle P \rangle$, conjugate P to Z -only form (by H or $S^\dagger H$ on affected qubits), measure computational basis bitstrings, map parity to ± 1 , and average to get a real number.

2.4 Choosing an ansatz

- **UCCSD.** Accurate chemistry-inspired circuit; depth roughly $\mathcal{O}(N^4)$ CNOTs where N is the number of spin-orbitals (integer).

Why: embodies physically relevant excitations; good accuracy when depth permits.

- **Hardware-Efficient (HEA).** Alternating layers of single-qubit rotations and entanglers; p layers (integer).

Why: adapts to device topology; shallow; good when depth/connectivity limit you.

- **Problem-inspired.** Structure that mirrors the target H .

Why: uses symmetries to shrink the parameter dimension $d = \dim(\theta)$ (integer) and reduce barren plateaus.

2.5 Expectation estimation and grouping

Energy estimator.

$$\hat{E}(\theta) = \sum_i h_i \widehat{\langle P_i \rangle},$$

where $\widehat{\langle P_i \rangle}$ is the sample mean of ± 1 outcomes after an appropriate basis change. The real number $\hat{E}(\theta)$ approximates $E(\theta)$.

Grouping. Partition $\{P_i\}$ into commuting sets so one basis change serves many terms.

Why: fewer circuits at fixed precision.

Shot budgeting. With M shots (an integer) for a term, standard error scales like $1/\sqrt{M}$.

Why: to reach a target error in $\hat{E}(\theta)$, allocate **more shots** to high-variance groups and **fewer** to low-variance groups.

2.6 Classical optimisation

- **SPSA.** Two energy calls per step; stochastic gradient on θ (a real vector); robust for many parameters.
- **COBYLA.** About $d + 1$ calls per step where $d = \dim(\theta)$; gradient-free; fast for small d .
- **Param-shift / QN-SR1.** Analytic-gradient style when available.

Why these choices. They trade off shot cost, noise robustness, and convergence speed based on your ansatz and backend.

2.7 Hybrid loop

Let θ be a real vector, E a real scalar, and `max_iter` an integer.

python

 Copy code

```
 $\theta$  =  $\theta_{\text{init}}$ 
for iter in range(max_iter):
    E = estimate_energy( $\theta$ )      # quantum: measure  $\langle P_i \rangle$  and sum
     $\theta$  = classical_update( $\theta$ , E) # classical: pick new parameters
    if converged(E): break
```

Why hybrid. Quantum is good at sampling $\langle P_i \rangle$. Classical is good at navigating noisy landscapes $E(\theta)$.

2.8 Example: Qiskit Nature workflow (H₂)

- Driver → integrals. Produce h_{pq}, h_{pqrs} (reals).
- Mapper → qubits. Build $H = \sum_i h_i P_i$.
- Ansatz → circuit. Prepare $|\psi(\theta)\rangle$ via $U(\theta)$.
- Estimator → energy. Compute $E(\theta)$.
- Optimiser → update. Produce a new θ .

Why this pipeline. It automates chemistry-to-qubits while making each mathematical object explicit and tunable.

3) Emerging ideas: SQD and SKQD

- SQD. Model noisy expectation values $\widehat{\langle P_i \rangle}$ as stochastic processes to stabilise gradients in θ . **Why:** fewer re-runs for similar precision.
- SKQD. Learn state/observable trajectories to interpolate estimates and cut shots M . **Why:** better sample efficiency with limited hardware time.

4) AI / DS applications

- Chemistry screening. Use energies $E(\theta)$ as labels or filters.
- Ising optimisation. Encode costs in H_{Ising} and minimise $E(\theta)$.
- Feature construction. Low-energy states of designed H act as informative embeddings.

Why: VQE turns domain objectives into measurable quantities you can optimise under hardware constraints.

5) Mini-exercises

1. **Measure $X_1 Y_2$.** Show that applying $R_X(-\pi/2)$ on qubit 1 and $R_Y(\pi/2)$ on qubit 2 turns measurement of $X_1 Y_2$ into a Z -basis measurement; π is the real constant $3.14159 \dots$
2. **UCC single excitation.** With $T = \theta(a_0^\dagger a_2 - a_2^\dagger a_0)$ where θ is real, map T to Pauli strings and write $e^{-i\theta T}$ using basis changes, a CNOT chain, one $R_Z(\theta)$, and uncomputation.
3. **HEA vs UCCSD.** On 4 qubits, compare convergence $E(\theta)$ for SPSA vs COBYLA with matched shot budgets.
4. **Shot budget.** With term variance $\sigma^2 \approx 0.25$ (real) and 100 commuting groups (integer), estimate shots M per group to reach total energy error $\leq 10^{-3}$ Hartree (real).
5. **Readout mitigation.** Given a 2×2 calibration matrix A (reals), show how corrected counts $c' = A^{-1}c$ yield $\langle Z \rangle = (c'_0 - c'_1)/(c'_0 + c'_1)$ (real).

6) FAQ (focused on “why”)

- **Why UCCSD depth grows.** Long Pauli strings from fermion mappings and Trotter steps expand CNOT count with the number of spin-orbitals N .
- **Why mix ansätze.** Combine physical insight with hardware-aware layers to keep $\dim(\theta)$ manageable and reduce barren plateaus.
- **Why not exact eigenstate count.** VQE needs only H and a symmetry sector; it measures and minimises $E(\theta)$ directly.
- **Why this stopping rule.** Use $|E(\theta_t) - E(\theta_{t-1})| < \varepsilon$ with tolerance $\varepsilon > 0$ to reflect stability under noise.

7) Summary

- **What:** Minimise $E(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle$ by alternating quantum measurements and classical updates.
- **How:** Map models (Ising or chemistry) to Pauli sums $H = \sum_i h_i P_i$; select $U(\theta)$; group terms; measure; optimise.
- **Why:** Ising captures discrete decisions; Pauli strings are native to qubits for both preparation $e^{-i(\theta/2)P}$ and measurement of $\langle P \rangle$; shallow circuits and grouping control noise and runtime while preserving a variational upper bound $E(\theta) \geq E_0$.